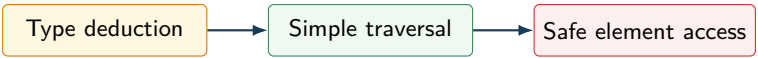
Object Oriented Programming System

Course Code: 25CS301

Lecture 5

Modern C++ Basics: `auto` and Range-Based `for`



B.Tech III Semester
Suggested Duration: 60 Minutes

Lecture Roadmap

- **Lecture Positioning**
- `auto` Keyword
- Range-Based `for` Loop
- Arrays and Vectors
- Integrated Beginner Programs
- Common Mistakes and Debugging
- Exam and Viva Preparation
- Lecture Summary

Where Lecture 5 Fits in Unit 1

Previous Lecture

Lecture 4 covered inline functions, function overloading, default arguments, and C-style variadic functions.

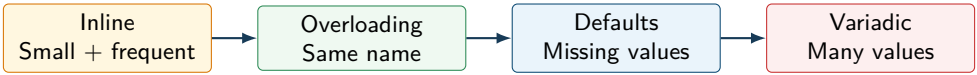
Today's Lecture

We complete Unit 1 with the `auto` keyword and range-based `for` loop, using arrays, strings, and vectors.

Why It Matters for OOP

These features make later class-based and generic programs shorter, easier to read, and less dependent on repeated type names or index bookkeeping.

Quick Recap of Lecture 4



Bridge to Lecture 5

Lecture 4 improved how functions receive and process input. Lecture 5 improves how local types are written and how stored data is traversed.

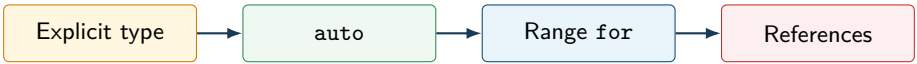
Learning Objectives

By the end of this lecture, students will be able to:

- explain why `auto` was introduced in modern C++;
- predict the type deduced from a clear initializer;
- distinguish `auto`, `auto&`, and `const auto&`;
- write range-based loops for arrays, strings, and vectors;
- modify collection elements safely using references;
- choose between a range-based loop and a traditional indexed loop;
- diagnose common type-deduction and traversal mistakes.

Beginner-Friendly Progression

Start with one initialized variable. Then deduce several simple types. Next traverse a small array. Only after that, use references and vectors.



Lecture Roadmap

- Lecture Positioning
- **`auto` Keyword**
- Range-Based `for` Loop
- Arrays and Vectors
- Integrated Beginner Programs
- Common Mistakes and Debugging
- Exam and Viva Preparation
- Lecture Summary

Why Was `auto` Introduced?

Problem

C++ types can become long or repetitive, especially when templates and standard-library collections are used.

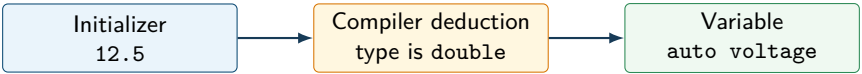
Modern C++ idea

If the initializer already tells the compiler the exact type, the programmer may ask the compiler to write that type automatically.

Important

`auto` improves convenience; it does not make C++ dynamically typed.

Meaning of `auto`



Definition

`auto` is a placeholder type specifier. The compiler determines the variable's type from its initializer at compile time.

Syntax of auto

General syntax

```
auto variable_name = initializer;
```

Examples

```
auto age = 18;
```
deduces `int`

```
auto marks = 91.5;
```
deduces `double`

```
auto grade = 'A';
```
deduces `char`

Rule

A variable declared with `auto` needs an initializer because deduction needs an expression to inspect.

Program 1: Simple Type Deduction

```cpp
#include <iostream>
using namespace std;

int main()
{
    auto count = 5;
    auto price = 49.75;
    auto grade = 'A';
    auto ready = true;

    cout << count << ' ' << price << ' '
         << grade << ' ' << ready;
    return 0;
}
```

Output

```
5 49.75 A 1
```

What Type Is Deduced?

Declaration	Deduced type	Reason
`auto n = 10;`	`int`	integer literal
`auto x = 3.5;`	`double`	decimal literal
`auto ch = 'Z';`	`char`	character literal
`auto flag = false;`	`bool`	Boolean literal
`auto text = "OOPS";`	`const char*`	string literal decays to pointer

Reading strategy

Read the initializer first. Then ask: "What type does this expression produce?"

auto Requires an Initializer

```
auto total = 0;        // Correct: int
auto average = 0.0;    // Correct: double

auto reading;          // Error: no initializer
reading = 18.5;
```

Why the last declaration fails

The later assignment cannot help the earlier declaration. The compiler must know the variable's type when the declaration is compiled.

Correct alternatives

`double reading;` or `auto reading = 18.5;`

Deduction Does Not Change Expression Rules

```
auto a = 5 / 2;       // int, value 2
auto b = 5.0 / 2;     // double, value 2.5
auto c = 5 / 2.0;     // double, value 2.5
```

Key idea

C++ evaluates the initializer using normal arithmetic rules first. `auto` then deduces the type of that result.

Common misconception

`auto` does not automatically choose the most precise type the programmer might have intended.

auto and const

```cpp
const int limit = 100;

auto copy = limit;          // int: top-level const removed
const auto fixed = limit;   // const int

copy = 120;                 // valid
// fixed = 120;             // error: read-only
```

Meaning

Plain `auto` creates a new value. Add `const` when the new variable itself must remain read-only.

auto& Preserves a Reference Relationship

```cpp
int score = 80;

auto copy = score;          // independent int
auto& alias = score;        // int&
const auto& view = score;   // const int&

copy = 90;                  // score remains 80
alias = 95;                 // score becomes 95
// view = 70;               // error: read-only reference
```

Remember

The symbols after `auto` express whether the program needs a copy, a writable alias, or a read-only view.

Choose the Form by Intent

Declaration	Relationship	Typical intention
`auto x = value`	independent copy	local calculation
`auto& x = value`	writable reference	modify original
`const auto& x = value`	read-only reference	observe without copying

Use the simplest declaration that accurately communicates what the code will do.

When `auto` Improves Code

- the initializer makes the type immediately obvious;
- an explicit type would repeat information already written on the right;
- a standard-library iterator or template-generated type is long;
- the exact type may change, but the operation should remain the same;
- range-based loops should follow the collection's element type.

Good use

`auto average = total / count;` is clear when `total` and `count` are already well named and correctly typed.

When an Explicit Type Is Clearer

Prefer an explicit type when it communicates an important contract:

- required precision, such as `double`;
- a narrow numeric range;
- signed vs unsigned behavior;
- ownership or pointer intent;
- an API boundary that students must understand.

Balanced rule

Do not replace every type name with `auto`. Use it when deduction makes the code easier to understand.

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- **Range-Based `for` Loop**
- Arrays and Vectors
- Integrated Beginner Programs
- Common Mistakes and Debugging
- Exam and Viva Preparation
- Lecture Summary

Need for a Range-Based `for` Loop

Traditional traversal

An indexed loop needs initialization, a boundary condition, an increment, and repeated element lookup.

Modern traversal

When the task is simply "process every element," a range-based loop states that intention directly.

Introduced in C++11

Range-based `for` works naturally with built-in arrays and standard collections such as `vector` and `string`.

Syntax of Range-Based for

General syntax

```
for (declaration : range) { statements }
```

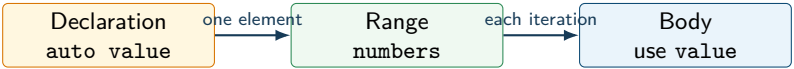
Example

```
for (auto value : numbers) { cout « value; }
```

Read it aloud

"For each value in numbers, execute the loop body."

Anatomy of a Range Loop



One iteration

The loop variable is created or rebound for the current element, and then the body executes.

Program 2: Traverse a Built-In Array

```cpp
#include <iostream>
using namespace std;

int main()
{
    int values[] = {10, 20, 30, 40};

    for (int value : values)
        cout << value << ' ';

    return 0;
}
```

Output

```
10 20 30 40
```

Dry Run: Array Traversal

Iteration	Current element	Output so far
1	10	10
2	20	10 20
3	30	10 20 30
4	40	10 20 30 40

Observation

No index is needed because the algorithm only needs each value in order.

Program 3: Traverse a Vector with auto

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<double> readings{18.5, 19.0, 20.0};

    for (auto value : readings)
        cout << value << '\n';

    return 0;
}
```

Deduction

On every iteration, `value` is deduced as `double`.

A String Is Also a Range of Characters

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string course = "OOPS";

    for (auto letter : course)
        cout << letter << ' ';
}
```

Output

```
O O P S
```

What a Range Loop Does Not Give Directly

- the current numeric index;
- a custom step size such as 2;
- a selected interval such as elements 3 through 7;
- automatic access to neighboring elements;
- reverse traversal unless the range itself is reversed.

Decision rule

Use a traditional indexed loop when the index is part of the problem. Use a range loop when the element itself is enough.

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- Range-Based `for` Loop
- **Arrays and Vectors**
- Integrated Beginner Programs
- Common Mistakes and Debugging
- Exam and Viva Preparation
- Lecture Summary

Plain auto Reads a Copy

```cpp
vector<int> marks{60, 70, 80};

for (auto mark : marks)
    mark += 5;

// marks is still {60, 70, 80}
```

Reason

Each iteration copies one element into `mark`. Changing the copy does not change the vector.

auto& Modifies the Actual Element

```cpp
vector<int> marks{60, 70, 80};

for (auto& mark : marks)
    mark += 5;

// marks is now {65, 75, 85}
```

Reason

`mark` is a reference to the current vector element, not a temporary copy.

const auto& Gives a Read-Only View

```
vector<string> names{"Asha", "Kabir", "Meera"};

for (const auto& name : names)
    cout << name << '\n';
```

Benefits

- avoids copying each string;
- prevents accidental modification;
- follows any future change to the element type.

Value, Reference, and Const Reference

Loop declaration	Effect	Best use
`auto x`	copies element	small values, local work
`auto& x`	aliases element	modify collection
`const auto& x`	read-only alias	inspect larger objects

Ask first: "Should this loop change the original collection?"

Traditional Loop vs Range-Based Loop

Traditional indexed loop	Range-based loop
gives an index	gives an element
manual boundary condition	range controls boundaries
useful for selected positions	useful for every element
supports custom step sizes	advances one element at a time
more syntax	clearer for simple traversal

Choose an Indexed Loop When Position Matters

```cpp
vector<int> marks{72, 81, 69};

for (size_t i = 0; i < marks.size(); ++i)
    cout << "Subject " << i + 1
         << ": " << marks[i] << '\n';
```

Why not a simple range loop?

The output needs both the value and its position. The index is part of the required information.

Empty Ranges Are Safe to Traverse

Example

If a vector contains zero elements, the body of a range-based loop executes zero times.

No special boundary condition

The loop does not access an invalid first element because it checks the range's beginning and end before iteration.

But the surrounding calculation may still need a check

An average calculation must avoid division by `values.size()` when the vector is empty.

Nested Range Loops for a Two-Dimensional Array

```cpp
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

for (const auto& row : matrix)
{
    for (auto value : row)
        cout << value << ' ';
    cout << '\n';
}
```

Why const auto& row?

The reference preserves each row as an array that can itself be traversed without copying it.

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- Range-Based `for` Loop
- Arrays and Vectors
- **Integrated Beginner Programs**
- Common Mistakes and Debugging
- Exam and Viva Preparation
- Lecture Summary

Program 4: Total and Average of Readings

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<double> readings{18.5, 19.0, 20.0, 21.5};
    auto total = 0.0;

    for (auto value : readings)
        total += value;

    auto average = total / readings.size();
    cout << "Total = " << total << '\n';
    cout << "Average = " << average;
}
```

Dry Run: Total and Average

Current value	Total before	Total after
18.5	0.0	18.5
19.0	18.5	37.5
20.0	37.5	57.5
21.5	57.5	79.0

Output

```
Total = 79     Average = 19.75
```

Program 5: Apply Grace Marks

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> marks{68, 75, 82};

    for (auto& mark : marks)
        mark += 2;

    for (const auto& mark : marks)
        cout << mark << ' ';
}
```

Output

```
70 77 84
```

Program 6: Student Marks with a Class

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Student
{
public:
    string name;
    vector<int> marks;

    void display()
    {
        cout << name << ": ";
        for (auto mark : marks) cout << mark << ' ';
    }
};
```

Using the Student Object

```
int main()
{
    Student s;
    s.name = "Asha";
    s.marks = {81, 76, 89};

    s.display();
    return 0;
}
```

Output

```
Asha:  81 76 89
```

OOP connection

The object stores its state, while the member function uses a range loop to present that state clearly.

Mini Case Study: Laboratory Calibration

A lab stores several sensor readings in a vector.

1. Use `auto` to store the calibration offset `0.5`.
2. Use `auto&` to add the offset to every reading.
3. Use `const auto&` to display adjusted readings.
4. Use an indexed loop only if each reading must be printed with its position.

This one problem tests type deduction, traversal, mutation, and loop selection together.

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- Range-Based `for` Loop
- Arrays and Vectors
- Integrated Beginner Programs
- **Common Mistakes and Debugging**
- Exam and Viva Preparation
- Lecture Summary

Common Mistake 1: Uninitialized `auto`

```cpp
auto value;          // Wrong

auto value = 0;      // Correct: int
double value = 0.0;  // Correct: explicit double
```

Reason

The compiler cannot deduce a type without an initializer.

Common Mistake 2: Unexpected Integer Division

```
auto average1 = 7 / 2;     // 3, type int
auto average2 = 7.0 / 2;   // 3.5, type double
```

Debugging question

What type do the operands have before `auto` performs deduction?

Fix

Make at least one operand floating-point when a decimal result is required.

Common Mistake 3: Modifying a Copy

```cpp
for (auto mark : marks)
    mark += 5;        // Does not modify marks

for (auto& mark : marks)
    mark += 5;        // Modifies marks
```

Check the declaration

If changes disappear after the loop, verify whether the loop variable is a copy or a reference.

Common Mistake 4: Iterating Over One Element

```
1 vector<int> values{10, 20, 30};
2
3 for (auto x : values[0]) { }  // Wrong: values[0] is int
4 for (auto x : values)    { }  // Correct: values is a range
```

Reason

A range-based loop needs something with a beginning and an end, not one scalar value.

Debugging Checklist

Before blaming the compiler, check:

- Does every `auto` declaration have an initializer?
- What type does the initializer expression actually produce?
- Is integer division occurring?
- Does the loop need a copy or a reference?
- Is a read-only loop declared with `const auto&` where appropriate?
- Is the expression after the colon a real range?
- Does the algorithm need the index?
- Could the range be empty before division or indexing?

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- Range-Based `for` Loop
- Arrays and Vectors
- Integrated Beginner Programs
- Common Mistakes and Debugging
- **Exam and Viva Preparation**
- Lecture Summary

Important Theory Questions

1. Define `auto`. Explain compile-time type deduction with examples.
2. Why must an `auto` variable have an initializer?
3. Differentiate `auto`, `auto&`, and `const auto&`.
4. Explain the syntax and working of a range-based `for` loop.
5. Compare a traditional indexed loop with a range-based loop.
6. Write a program to traverse and modify a vector using range-based loops.
7. Explain two common errors involving `auto` and range traversal.

Viva Questions

1. Is `auto` type deduction performed at runtime?
2. What does `auto x = 5 / 2;` deduce?
3. Can `auto x;` compile? Why?
4. Why does plain `auto` often create a copy?
5. When should `auto&` be used in a range loop?
6. Why is `const auto&` useful for strings?
7. Can a range loop directly provide an index?
8. What happens when the range is empty?

MCQs: Set 1

1. `auto x = 10.5;` deduces:
   A. `int` B. `float` C. `double` D. `char`
2. Which declaration is invalid?
   A. `auto a = 5;` B. `auto b;` C. `auto c = true;` D. `auto d = 'A';`
3. Which loop can modify vector elements?
   A. `for(auto x:v)` B. `for(auto& x:v)` C. `for(const auto& x:v)` D. none

Answers

1-C, 2-B, 3-B

MCQs: Set 2

1. Range-based `for` is best when:
   A. every element is processed B. an index is required C. a step of 2 is required D. only one element is used
2. `auto y = 9 / 4;` produces:
   A. 2.25 B. 2 C. 3 D. error
3. Which is best for read-only traversal of strings?
   A. `auto` B. `auto&` C. `const auto&` D. `int`

Answers

1-A, 2-B, 3-C

Predict the Output

```
int score = 50;
auto a = score;
auto& b = score;

a += 10;
b += 20;

cout << score << ' ' << a << ' ' << b;
```

Options

A. 50 60 70
B. 70 60 70
C. 80 80 80
D. compilation error

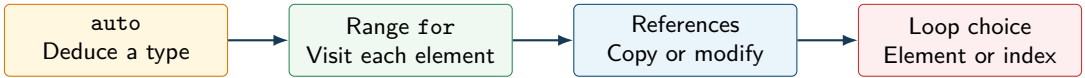
Answer: B

`a` is a copy. `b` aliases `score`; therefore both `score` and `b` become 70.

Lecture Roadmap

- Lecture Positioning
- `auto` Keyword
- Range-Based `for` Loop
- Arrays and Vectors
- Integrated Beginner Programs
- Common Mistakes and Debugging
- Exam and Viva Preparation
- **Lecture Summary**

One-Slide Summary



Lecture 5 completes Unit 1 by making types and traversal more expressive while preserving compile-time checking.

Assignment

1. Write a program that stores five temperatures in a vector and calculates their average using `auto` and a range loop.
2. Modify every temperature by a calibration offset using `auto&`.
3. Display the adjusted values using `const auto&`.
4. Print each adjusted value with its position using an indexed loop.
5. Explain in five lines why the three loops use different declarations.

Submit

C++17 source code, two labelled test runs, and a short concept explanation.

Unit 1 Closure and Next Step

Unit 1 now connects

programming paradigms, C++ structure, classes and objects, references, parameter passing, advanced function features, `auto`, and range-based `for`.

Next step

Apply these language foundations to stronger class design, where data and operations are organized with clearer access control and responsibility.

Preparation

Revise classes, objects, member functions, references, and the copy/reference behavior demonstrated today.

Thank You

Questions?

Remember: Use modern syntax when it makes program intent clearer.